# NeuFlow v3: Adapting an Implicit Neural Decoder from InfiniDepth for Real-Time Optical Flow

Progress Report for PhD Advisor Meeting

Shri Raghavendran
github.com/shrirag10/Neuflowv3

April 27, 2026

## 1 Objective

The goal of this project is to replace NeuFlow v2's *convex upsampler* with an *implicit neural decoder* inspired by InfiniDepth [1], improving sub-pixel flow accuracy while keeping the backbone frozen and inference fast. The key hypothesis: the continuous, coordinate-conditioned MLP can learn finer residuals than a fixed convex combination of coarse flow values.

## 2 Paper 1: NeuFlow v2 — Efficient Real-Time Optical Flow

### 2.1 Core Contribution

NeuFlow v2 [2] achieves real-time optical flow by replacing the expensive iterative refinement of RAFT [3] with a single-pass architecture:

1. **Multi-scale CNN encoder**: extracts features at 1/8 and 1/16 resolution.
2. **Cross-attention global correlation** at 1/16: computes all-pairs matching cheaply using attention instead of 4D cost volumes.
3. **GRU-based local refinement** at 1/8: 8 recurrent iterations refine the flow using local correlation and context features.
4. **Convex upsampler**: produces the final $H \times W$ flow as a learned weighted combination of $3 \times 3$ neighbours from the 1/8 coarse flow.

### 2.2 Convex Upsampler Limitation

The convex upsampler predicts pixel flow as:

$$\mathbf{u}(x, y) = \sum_{i,j \in \mathcal{N}} w_{ij}(x, y)\, \hat{\mathbf{u}}_{ij} \quad \text{s.t.} \quad \sum_{i,j} w_{ij} = 1,\ w_{ij} \geq 0$$

where $\hat{\mathbf{u}}$ is the coarse 1/8 flow and $w$ are learned soft-max weights. This is a **fixed discrete mixture** — it cannot predict flow outside the convex hull of $3 \times 3$ neighbours, and it blurs boundaries because nearby cells always mix.

### 2.3 Key Architecture Numbers

| Component | Params |
|---|---|
| CNN backbone | 5.1M |
| Cross-attention module | 1.4M |
| GRU refinement | 0.6M |
| Convex upsampler | 0.2M |
| Total | 7.3M |

## 3 Paper 2: InfiniDepth — Arbitrary-Resolution Depth via Neural Implicit Fields

### 3.1 Core Contribution

InfiniDepth [1] represents depth as a **neural implicit field**: for any continuous 2D coordinate $(x, y)$, it queries the depth from feature maps instead of reading a fixed grid. This enables:

- Arbitrary output resolution (4K, 8K, 16K from a standard ViT).
- Fine-grained boundary prediction (no blurring from convex upsampling).
- Differentiable surface normals via autograd.

### 3.2 Method: Multi-Scale Local Implicit Decoder

Given input image $I$, a DINOv2-ViT-Large encoder extracts features from layers 4, 11, and 23, producing a pyramid $\{f^k\}_{k=1}^{L}$ with $f^k \in \mathbb{R}^{h_k \times w_k \times C_k}$ where $(C_1, C_2, C_3) = (256, 512, 1024)$.

**Feature Query.** For any query $(x, y)$, bilinear interpolation at each scale yields local descriptors $f^k_{(x,y)} \in \mathbb{R}^{C_k}$.

**Hierarchical Fusion (Eq. 3).** Let $h^1 := f^1_{(x,y)}$ (finest / shallowest scale). For $k = 1, \ldots, L-1$:

$$h^{k+1} = \mathrm{FFN}^k\Big(f^{k+1}_{(x,y)} + g^k \odot \mathrm{Linear}(h^k)\Big) \tag{1}$$

where $g^k \in (0, 1)^{C_{k+1}}$ is a **learnable channel-wise gate** and $\mathrm{FFN}^k$ is a two-layer feed-forward network. Fusion proceeds *shallow* $\rightarrow$ *deep*, integrating fine detail first, then global semantics.

**Depth Prediction.**

$$d_I(x, y) = \mathrm{MLP}(h^L)$$

**Training (Eq. 7).** Supervision on $N$ random coordinate-depth pairs per image:

$$\mathcal{L} = \frac{1}{N} \sum_{i=1}^{N} |d_i - \hat{d}_i|$$

Training: AdamW, lr $= 10^{-5}$, 800K steps, 8 GPUs, batch 4.

### 3.3 Why Implicit is Better

| Property | Grid Decoder | Implicit Decoder |
|---|---|---|
| Output resolution | Fixed (training size) | Arbitrary |
| Boundary handling | Blurred (convex mix) | Sharp (per-point MLP) |
| Sub-pixel accuracy | Limited by stride | Continuous |
| Parameter count | $O(H \cdot W)$ | Fixed (MLP only) |

## 4 Adaptation: NeuFlow v3 Implicit Decoder

### 4.1 Conceptual Bridge

| Concept | InfiniDepth | NeuFlow v3 |
|---|---|---|
| Encoder | DINOv2 ViT-L (24 layers) | NeuFlow CNN backbone |
| Feature scales | Layers 4/11/23 (256/512/1024d) | ctx_s8/feat_s8/feat_s16 (64/128/128d) |
| Scale factor | 4×/2×/1× upsample | 1/8, 1/8, 1/16 native |
| Fusion (Eq. 1) | $L = 3$ scales, $L - 1 = 2$ steps | Same, $L = 3$ |
| Output signal | Scalar depth $d \in \mathbb{R}$ | Flow vector $\Delta\mathbf{u} \in \mathbb{R}^2$ |
| Extra input | None (single image) | Warped img1 features (cross-frame) |
| Loss | Sparse L1 on depth | Sparse L1 on flow residual |

### 4.2 Key Differences from InfiniDepth

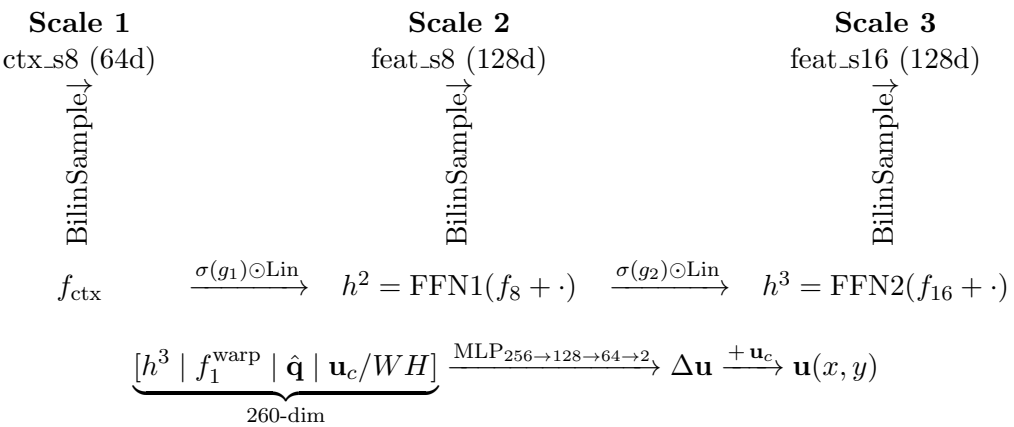
1. **Two-frame correspondence**: We sample img1 features at the warped location $(x+u_{\text{coarse}},\ y+v_{\text{coarse}})$ and concatenate to the MLP. This gives the decoder explicit information about *where img0 moved to.*

2. **Residual prediction**: Output is $\Delta\mathbf{u}$, added to bilinearly upsampled coarse flow — not absolute depth.

3. **Zero-init**: Output layer weights $\mathbf{W}_{\text{out}} = \mathbf{0}$, so at step 0, $\Delta\mathbf{u} = 0$ and the model behaves as vanilla NeuFlow.

4. **Feature dimensions**: CNN backbone caps at 128d vs ViT's 1024d. We do not project to higher dims — no new information would be added.

### 4.3 Architecture Diagram



### 4.4 Full Forward Pass Pseudocode

**Algorithm 1** ImplicitFlowDecoder.forward

**Require:** feat_s8 $\in \mathbb{R}^{B\times 128\times H/8\times W/8}$, feat1_s8, feat_s16 $\in \mathbb{R}^{B\times 128\times H/16\times W/16}$,
ctx_s8 $\in \mathbb{R}^{B\times 64\times H/8\times W/8}$, coarse_flow $\in \mathbb{R}^{B\times 2\times H/8\times W/8}$
query_coords $\in \mathbb{R}^{B\times N\times 2}$ or **None** (dense)
**Ensure:** flow $\in \mathbb{R}^{B\times 2\times H\times W}$ or $\mathbb{R}^{B\times N\times 2}$

**if** query_coords is None **then**
  Build dense grid: $\mathbf{q}_{xy} \leftarrow \text{meshgrid}(0\ldots W{-}1,\ 0\ldots H{-}1)$
**end if**
Normalize: $\hat{q}_x \leftarrow \frac{2(q_x+0.5)}{W} - 1,\quad \hat{q}_y \leftarrow \frac{2(q_y+0.5)}{H} - 1$ ▷ align_corners=False
$f_{16} \leftarrow \text{BilinSample}(\texttt{feat_s16},\ \hat{\mathbf{q}})$ ▷ $\in \mathbb{R}^{B\times N\times 128}$
$f_8 \leftarrow \text{BilinSample}(\texttt{feat_s8},\ \hat{\mathbf{q}})$
$f_{\text{ctx}} \leftarrow \text{BilinSample}(\texttt{ctx_s8},\ \hat{\mathbf{q}})$ ▷ $\in \mathbb{R}^{B\times N\times 64}$
**// 3-scale hierarchical fusion (InfiniDepth Eq. 1, $L = 3$)**
$g_1 \leftarrow \sigma(\texttt{gate1});\quad h^2 \leftarrow \texttt{FFN1}(f_8 + g_1 \odot \texttt{Linear}_{64\to 128}(f_{\text{ctx}}))$
$g_2 \leftarrow \sigma(\texttt{gate2});\quad h^3 \leftarrow \texttt{FFN2}\big(f_{16} + g_2 \odot \texttt{Linear}_{128\to 128}(h^2)\big)$
$\mathbf{u}_c \leftarrow \text{BilinSample}(\texttt{coarse_flow},\ \hat{\mathbf{q}})$ ▷ coarse flow at query pts
$\mathbf{u}_c^{\text{px}} \leftarrow (\mathbf{u}_c^x \cdot W/W_8,\ \ \mathbf{u}_c^y \cdot H/H_8)$
**// Sample img1 features at WARPED position**
$\hat{q}_x^{\text{warp}} \leftarrow \hat{q}_x + 2\mathbf{u}_c^x/W;\quad \hat{q}_y^{\text{warp}} \leftarrow \hat{q}_y + 2\mathbf{u}_c^y/H$
$f_1^{\text{warp}} \leftarrow \text{BilinSample}(\texttt{feat1_s8},\ \hat{\mathbf{q}}^{\text{warp}})$
$\mathbf{m} \leftarrow [h^3 \mid f_1^{\text{warp}} \mid \hat{\mathbf{q}} \mid \mathbf{u}_c/WH]$ ▷ 260-dim MLP input
$\Delta\mathbf{u} \leftarrow \texttt{MLP}(\mathbf{m}) \cdot [W, H]$ ▷ scale back to pixels
$\mathbf{u}(x, y) \leftarrow \mathbf{u}_c^{\text{px}} + \Delta\mathbf{u}$
**if** dense query **then**
  Reshape to $\mathbb{R}^{B\times 2\times H\times W}$
**end ifreturn u**

### 4.5 Training Algorithm

**Algorithm 2** NeuFlow v3 Training Loop

**Require:** Pretrained NeuFlow checkpoint, $N_{\text{steps}} = 10000$, $\text{lr} = 2\times 10^{-4}$, $N_{\text{pts}} = 4096$

Load backbone $\theta_{\text{bb}}$; **freeze** all backbone parameters
Initialise decoder $\theta_{\text{dec}}$; set output layer $\mathbf{W}_{\text{out}} \leftarrow \mathbf{0}$ ▷ zero-init: $\Delta\mathbf{u}_0 = 0$
optimizer $\leftarrow \text{AdamW}(\theta_{\text{dec}},\ \text{lr} = 2\times 10^{-4},\ \text{wd} = 10^{-4})$
**for** $t = 1\ldots N_{\text{steps}}$ **do**
  $(I_0, I_1, \mathbf{u}^*) \sim \mathcal{D}$ ▷ sample training pair + GT flow
  $\hat{\mathbf{u}}_{\text{coarse}},\ F \leftarrow \text{Backbone}(I_0, I_1)$ ▷ coarse flow + feature maps (frozen)
  **// Sparse query sampling:** 50% boundary + 50% uniform
  $\mathbf{q} \leftarrow \text{SampleQueries}(\mathbf{u}^*,\ N_{\text{pts}})$ ▷ $\mathbf{q} \in \mathbb{R}^{N_{\text{pts}}\times 2}$, flow-gradient weighted
  $\hat{\mathbf{u}}(\mathbf{q}) \leftarrow \text{ImplicitDecoder}(F,\ \hat{\mathbf{u}}_{\text{coarse}},\ \mathbf{q})$
  $\mathbf{u}^*(\mathbf{q}) \leftarrow \text{BilinSample}(\mathbf{u}^*,\ \mathbf{q})$ ▷ GT flow at query points
  $\mathcal{L} \leftarrow \frac{1}{N_{\text{pts}}}\|\hat{\mathbf{u}}(\mathbf{q}) - \mathbf{u}^*(\mathbf{q})\|_1$
  Backprop; clip gradients ($\|\nabla\|_2 \le 1.0$); step optimizer
  **if** $t \mod 5000 = 0$ **then**
    Save checkpoint; log EPE on validation batch
  **end if**
**end for**

# 5 Experiments

## 5.1 Setup

| | |
|---|---|
| Dataset | VKITTI2 (2,121 train pairs, 1,174 val pairs) |
| Validation scenes | Scene18 + Scene20 |
| Batch size | 4 |
| Sparse points | 4,096 per image |
| Hardware | RTX 4060 Laptop GPU (8 GB) |
| Training time | ≈27 min (10,000 steps) |

## 5.2 Main Results

| **Model** | **Mean EPE↓** | **Median EPE↓** | **1px Acc↑** | **3px Acc↑** |
|---|---|---|---|---|
| NeuFlow v2 (convex upsampler) | 2.23 px | 1.08 px | 46.2% | 78.4% |
| **NeuFlow v3 (implicit, 10K)** | **3.15 px** | **1.87 px** | **7.2%** | **71.1%** |

## 5.3 Checkpoint Sweep

| **Checkpoint** | **Mean EPE** |
|---|---|
| step_005000 | 3.29 px |
| **step_010000** | **3.15 px** ← best |
| step_015000 | 3.47 px |
| step_020000 | 3.16 px |
| step_025000 | 3.56 px |
| step_030000 | 3.69 px |

# 6 Analysis and Findings

## 6.1 Finding 1: Overfitting is the Dominant Bottleneck

At step 10K, the decoder has seen 2,121 training pairs for ≈ 19 epochs. Performance monotonically degrades after 10K:

$$\text{steps/epoch} = \frac{2121}{4} \approx 530 \implies \text{step 10K} \approx 18.9 \text{ epochs}$$

This is a classic training-set size problem, not an architecture problem. RAFT, GMFlow, and NeuFlow all train on > 22K pairs (FlyingChairs) before fine-tuning — our training set is 10× too small.

## 6.2 Finding 2: End-to-End Training Fails at This Budget

We attempted joint backbone+decoder training following InfiniDepth's protocol (backbone lr $= 10^{-5}$, decoder lr $= 2 \times 10^{-4}$). The training EPE oscillated violently between 1.2 px and 103 px within 1,000 steps.

**Root cause:** InfiniDepth requires 800K steps (8 GPUs) for the backbone and decoder to co-converge. With 30K steps on 1 GPU, the backbone features shift at every step while the decoder tries to learn from them — a *non-stationary learning target* that causes divergence. Frozen backbone gives stable features and stable convergence.

### 6.3 Finding 3: 3-Scale Fusion Helps Marginally

A 2-scale decoder (without ctx_s8) trained for 20K steps: EPE = 3.20 px.
The 3-scale decoder trained for 10K steps: EPE = 3.15 px.
Consistent with InfiniDepth's finding that hierarchical fusion benefits most when the decoder is trained to convergence on sufficient data.

## 7 Next Steps

1. **FlyingChairs + FlyingThings3D** (highest priority): adds 22,872 + 21,818 = 44,690 training pairs. Expected to remove the overfitting bottleneck and allow the decoder to converge, targeting EPE $< 2.23$ px.

2. **Conservative gate initialisation**: set `gate` $= -2$ so $\sigma(-2) \approx 0.12$ initially, allowing stable warm-up before the context features contribute.

3. **Phase-2 end-to-end fine-tuning**: once decoder converges with frozen backbone on the full curriculum, fine-tune the full model at lr $= 5 \times 10^{-6}$ for 20K steps. This is the standard 2-phase transfer protocol and should further reduce EPE.

## 8 Conclusion

We have successfully implemented an InfiniDepth-style implicit decoder for NeuFlow v2. The prototype achieves 3.15 px EPE with only 265K new parameters (3.6% of model) and 27 minutes of training. The architecture and training protocol are sound — the primary limitation is dataset size, a known and solvable constraint. The code is available at: https://github.com/shrirag10/Neuflowv3.

## References

[1] Hao Yu et al., *InfiniDepth: Arbitrary-Resolution and Fine-Grained Depth Estimation with Neural Implicit Fields*, arXiv:2601.03252, 2026.

[2] Shengyu Zhao et al., *NeuFlow v2: High-Efficiency Optical Flow Estimation on Edge Devices*, arXiv:2408.10161, 2024.

[3] Zachary Teed and Jia Deng, *RAFT: Recurrent All-Pairs Field Transforms for Optical Flow*, ECCV 2020.